BÀI THỰC HÀNH BUỔI 4

Vibe coding từng bước SQL Server + Sequelize

Nối tiếp đúng project backend-quan-ly-sinh-vien

Buổi trước bạn đã làm Backend quản lý sinh viên: xem danh sách, tìm theo mã và thêm mới. Dữ liệu còn nằm trong mảng của Service. Buổi này giữ project, đường dẫn API và cấu trúc phản hồi; chuyển nơi lưu dữ liệu sang SQL Server bằng Sequelize ORM.

Trước: Service → mảng trong RAM.
Hôm nay: Service → SinhVien Model → Sequelize/Tedious → SQL Server.
Bằng chứng: thêm SV003 bằng Postman, khởi động lại Node.js, vẫn tìm thấy SV003.

Nền kỹ thuật giữ nguyên

- pnpm 10; Express 5; TypeScript; package.json có type: module.
- ESM/NodeNext; tsx chạy phát triển; import nội bộ có đuôi .js.
- pnpm dev, pnpm build, pnpm start và cổng 5000.
- DTO bốn trường string: maSV, hoTen, email, maLop.

Bài chính chuyển các chức năng đã có sang CSDL; không tạo dự án mới. Quan hệ Lớp 1-N Sinh viên nằm ở phần mở rộng, sau khi bài chính chạy đúng. Không yêu cầu PUT, DELETE, frontend hoặc JWT.

1. Cách dùng tài liệu và AI

AI agent trong IDE	Cửa sổ chat
Mở project cũ; yêu cầu đọc file trước khi sửa.	Dán cây thư mục, package.json, tsconfig và file liên quan.
Xem file đã thay đổi và kết quả lệnh.	Tự tạo file tại đường dẫn được ghi; chép code AI trả về.
Không mặc định agent đã test SQL Server thật.	AI không tự thấy Database hoặc file trên máy.

Chu trình: đọc mục tiêu → gửi một prompt → xem file cần thay đổi → lưu → build → kiểm tra kết quả → đi tiếp. Cùng một cuộc trò chuyện giúp giữ bối cảnh, nhưng khi sửa vẫn gửi phiên bản file hiện tại.

Quy tắc cho mọi bước

• Chỉ sửa source trong src; không sửa JavaScript sinh trong dist.

• File cũ phải được ghép đầy đủ, giữ chức năng hiện có; không nối hai phiên bản server.ts.

• Lệnh pnpm chạy ở gốc có package.json. Script SQL có GO chạy trong SSMS.

• Không đổi pnpm sang npm, ESM sang CommonJS hay TypeScript sang .js.

• Không gửi .env/mật khẩu thật cho AI; dùng .env.example với giá trị mẫu.

• Không dùng DROP, sync force hoặc sync alter để chữa lỗi. Bảng tạo qua script SQL.

Điểm dừng kiểm tra:
Bạn biết công cụ AI có sửa file trực tiếp hay chỉ trả code, và biết cách đưa từng file vào đúng thư mục.

Vibe coding có trách nhiệm

Bạn quyết định yêu cầu, kiểm tra và giải thích được code. Một câu trả lời AI hoặc một lần build thành công chưa chứng minh API đọc/ghi đúng SQL Server.

2. Kiểm tra project buổi trước

1. Mở VS Code → Open Folder → backend-quan-ly-sinh-vien đang làm. Không chạy pnpm init lại.

2. Terminal ở thư mục có package.json: chạy node -v và pnpm -v. Nếu thiếu thư viện, chạy pnpm install.

3. Chạy pnpm build, sau đó pnpm dev. Thử GET /, GET /api/v1/sinh-vien và GET /api/v1/sinh-vien/SV001.

4. Sao lưu/commit phiên bản chạy đúng. Ghi nhận mọi sinh viên đã thêm trong RAM muốn giữ; có thể xuất GET danh sách thành JSON để đối chiếu. Dừng server trước khi sửa.

Chat: gửi package.json, tsconfig.json, server.ts, DTO, Service, Controller, Route, error.middleware.ts và AppError.ts. Agent: cho phép đọc đúng project này.

PROMPT - sao chép gửi AI

ROLE: Hướng dẫn tôi tiếp tục bài quản lý sinh viên, không tạo project mới.
CONTEXT: pnpm, Express 5, TypeScript ESM/NodeNext, tsx; import nội bộ .js. Bài trước có DTO ISinhVienDTO và API GET danh sách, GET theo maSV, POST thêm sinh viên. Dữ liệu còn ở mảng trong Service.
TASK: Đọc các file hiện tại; xác nhận tên hàm, cấu trúc phản hồi, validation, AppError và middleware. Lập danh sách file cần thêm/sửa để lưu SQL Server qua Sequelize 6. Chưa viết code.
CONSTRAINT: Giữ /api/v1/sinh-vien, maSV/hoTen/email/maLop, success/message/data, pnpm dev/build/start, GET / và middleware. Không chuyển sang ví dụ Products/Categories. Nếu có Repository đã làm ở phần mở rộng, giữ tầng đó và chỉ thay phần truy cập dữ liệu. Nêu đúng file còn thiếu trước khi sửa.

Điểm dừng kiểm tra:
Project cũ build được. Bạn xác định đủ chức năng đã làm; các bước sau phải giữ GET theo mã và POST, không chỉ chuyển mỗi GET danh sách sang Database.

3. Hợp đồng API phải giữ

Request	Kết quả cần giữ
GET /	200; success=true; message: Backend quản lý sinh viên đang hoạt động.
GET /api/v1/sinh-vien	200; success=true; message lấy danh sách; data là mảng.
GET /api/v1/sinh-vien/SV001	200; data là object, maSV=SV001.
GET mã không có	404; success=false; Không tìm thấy sinh viên.
POST /api/v1/sinh-vien hợp lệ	201; success=true; Thêm sinh viên thành công; data là sinh viên vừa lưu.
POST trùng mã	409; success=false; Mã sinh viên đã tồn tại.
POST sai dữ liệu / JSON hỏng	400; success=false; thông báo dễ hiểu, không lưu dữ liệu.
GET /khong-co	404; success=false; Không tìm thấy đường dẫn.

Quy tắc dữ liệu từ bài trước

Input phải là object, không null/array. Bốn trường bắt buộc là string; trim hai đầu. maSV đúng /^SV[0-9]{3}$/; hoTen và maLop không rỗng; email có dạng cơ bản tên@miền.phầnđuôi, không khoảng trắng. Không tự chuyển số thành chuỗi; chỉ lưu bốn trường DTO, bỏ trường thừa.

Điều thay đổi: các hàm Service truy cập SQL Server trở thành async. Controller phải await. Mã trùng được PK trong Database bảo vệ thêm. Restart Node.js không còn reset dữ liệu.

Bài chính chưa yêu cầu email duy nhất hoặc maLop phải tồn tại trong bảng lớp. Không tự thêm các quy tắc đó vào AI prompt. Giới hạn độ dài cột SQL ở trang sau phải được kiểm tra trước khi ghi.

4. Cây thư mục nâng cấp

```
backend-quan-ly-sinh-vien/
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
├── .env
├── .env.example
├── .gitignore
├── sql/
│   ├── 01-quan-ly-sinh-vien.sql     # them
│   ├── 02-seed.sql                 # them
│   └── 03-app-login.sql            # them
└── src/
    ├── server.ts                   # sua khoi dong
    ├── config/database.ts          # them
    ├── models/SinhVien.ts           # them
    ├── scripts/check-db.ts          # them
    ├── dtos/sinhVien.dto.ts         # giu DTO
    ├── services/sinhVien.service.ts # mang -> DB
    ├── controllers/sinhVien.controller.ts # them await
    ├── routes/sinhVien.routes.ts    # giu URL/method
    ├── middlewares/
    │   ├── logger.middleware.ts     # giu
    │   └── error.middleware.ts      # giu AppError
    └── utils/AppError.ts            # giu
```

node_modules và dist vẫn được sinh tự động. Không tạo app.ts hay một module sản phẩm mới. Nếu đã tự làm Repository theo phần mở rộng bài trước, thêm kết nối/Model vào đường truy cập đó, không bỏ Repository.

Bản đồ import ESM cần nhớ

```
// File src/models/SinhVien.ts:
import { sequelize } from "../config/database.js";
// File src/services/sinhVien.service.ts:
import { SinhVien } from "../models/SinhVien.js";
// File src/server.ts:
import { sequelize } from "./config/database.js";
```

File source vẫn có đuôi .ts; import nội bộ viết .js để JavaScript đã build chạy được với ESM/NodeNext. Không thêm đuôi kép .ts.js.

5. Thiết kế bảng SinhVien

Database bài học: **QuanLySinhVienDB**. Bảng: **dbo.SinhVien**. maSV là khóa chính chuỗi, không thêm id tự tăng vì bài trước tìm/nhận diện sinh viên theo maSV.

Cột SQL	Kiểu và ràng buộc	DTO
maSV	VARCHAR(5), PK, NOT NULL	string
hoTen	NVARCHAR(150), NOT NULL	string
email	NVARCHAR(254), NOT NULL	string
maLop	NVARCHAR(50), NOT NULL	string

NVARCHAR lưu tên tiếng Việt. NOT NULL không thay thế kiểm tra chuỗi rỗng hoặc định dạng email. Không đặt UNIQUE email vì bài trước chỉ quy định maSV không trùng.

Giới hạn mới do chọn kích thước cột

Ngoài validation cũ, Service kiểm tra hoTen không quá 150 ký tự, email không quá 254 và maLop không quá 50. Vượt giới hạn trả 400 trước khi ghi; không cắt chuỗi tự động. Đây là ràng buộc bổ sung được công bố trong bài này, không giả định đã có ở bài trước.

Seed giữ đúng ví dụ cũ

```
SV001 | Minh Nhật | minhnhat@example.com | 25CT114
SV002 | Lan Anh   | lananh@example.com   | 25CT114
```

Điểm dừng kiểm tra:
Bạn nói được maSV là PK và vì sao không dùng id số thay maSV. Bài chính có một bảng; quan hệ Lớp–Sinh viên được thêm ở cuối sau khi API hiện có hoạt động.

Database và ORM khác gì?

SQL Server lưu bản ghi. Sequelize là thư viện trong Node.js ánh xạ Model sang bảng; Tedious là driver kết nối. DTO mô tả dữ liệu trao đổi; Model mô tả dữ liệu được lưu. Hai phần có thể cùng bốn trường nhưng khác vai trò.

6. Chuẩn bị SQL Server local

1. Xác nhận SQL Server Database Engine đang chạy. SSMS chỉ là công cụ quản lý, không thay thế Engine. Kết nối SSMS vào đúng instance.

2. Mở SQL Server Configuration Manager → SQL Server Network Configuration → Protocols for đúng instance → bật TCP/IP.

3. TCP/IP Properties: với Listen All=Yes, vào IP Addresses → IPAll; xóa TCP Dynamic Ports (kể cả 0), nhập TCP Port=1433.

4. Restart đúng Engine trong SQL Server Services. Nếu cổng bị instance khác chiếm, chọn cổng trống và ghi lại để dùng nhất quán.

5. Thử SSMS với server name tcp:127.0.0.1,1433 (hoặc cổng đã chọn).

6. Server Properties → Security → SQL Server and Windows Authentication mode để dùng SQL login. Restart sau khi đổi.

Bài này kết nối theo host + port; không vừa đặt instanceName vừa port. Đăng nhập SSMS bằng tên máy\SQLEXPRESS chưa chứng minh cổng TCP 1433 hoạt động.

PROMPT - sao chép gửi AI

Tôi dùng SQL Server local trên Windows. Instance: [điền]. Cổng: [điền]. Khi kết nối SSMS bằng tcp:127.0.0.1,[cổng] tôi nhận lỗi: [dán]. Hãy hướng dẫn kiểm tra Engine, đúng instance, TCP/IP, IPAll, restart và SQL Authentication từng bước. Chưa sửa Backend, không đề xuất xóa Database hoặc mở firewall rộng để chữa lỗi local.

Điểm dừng kiểm tra:
SSMS kết nối được theo TCP vào đúng instance. Nếu chưa được, xử lý bước này trước khi tạo cấu hình Sequelize.

7. Tạo bảng và dữ liệu mẫu bằng AI

PROMPT - sao chép gửi AI

Tạo hai file SQL Server cho bài quản lý sinh viên:
1. sql/01-quan-ly-sinh-vien.sql: tạo QuanLySinhVienDB và dbo.SinhVien. maSV VARCHAR(5) PRIMARY KEY NOT NULL, hoTen NVARCHAR(150) NOT NULL, email NVARCHAR(254) NOT NULL, maLop NVARCHAR(50) NOT NULL. Không thêm id, timestamps hoặc UNIQUE email.
2. sql/02-seed.sql: seed SV001, Minh Nhật, minhnhat@example.com, 25CT114; SV002, Lan Anh, lananh@example.com, 25CT114. Dùng N'...' cho chuỗi Unicode. Chỉ INSERT mã chưa tồn tại để chạy lại không trùng; không UPDATE ghi đè dữ liệu đã thay đổi. Có SELECT cuối file ORDER BY maSV.
Nếu Database/bảng đã có, kiểm tra schema thay vì xóa. Không DROP. Đưa đầy đủ từng file, giải thích maSV là PK và cách chạy theo thứ tự trong SSMS. Chưa viết Model hoặc Backend.

1. Ở gốc project, tạo thư mục sql và lưu đúng hai file.

2. Mở SSMS bằng tài khoản có quyền tạo DB → New Query → chạy file 01. Nếu Database đã có, đối chiếu schema; không xóa dữ liệu cũ.

3. Chạy file 02 và xem SELECT. Với DB mới có hai sinh viên.

4. Refresh Object Explorer → QuanLySinhVienDB → Tables → dbo.SinhVien. Kiểm tra Columns và PK.

```
USE QuanLySinhVienDB;
SELECT maSV, hoTen, email, maLop
FROM dbo.SinhVien ORDER BY maSV;
```

Điểm dừng kiểm tra:
Đủ SV001 và SV002; chữ Minh Nhật/Lan Anh đúng. Chạy lại seed không tạo bản ghi trùng và không phục hồi tên cũ nếu bạn đã sửa tên.

GO phân tách batch của SSMS. Không chạy file .sql bằng node/tsx, không đưa nguyên script có GO vào sequelize.query.

8. Tạo SQL login cho ứng dụng

PROMPT - sao chép gửi AI

Tạo sql/03-app-login.sql chạy bằng tài khoản quản trị:
SQL login sv_app, user sv_app trong QuanLySinhVienDB, chỉ GRANT SELECT, INSERT ON dbo.SinhVien vì project hiện có GET và POST. Không dùng sa, sysadmin hoặc db_owner.
Dùng mật khẩu placeholder, giải thích thay bằng mật khẩu local trước khi chạy. Nếu login/user đã có, hướng dẫn kiểm tra và dùng lại, không xóa. Script dùng đúng USE master/USE QuanLySinhVienDB và GO.
Hướng dẫn thử SSMS bằng SQL Server Authentication với sv_app, server tcp:127.0.0.1,1433 và chọn QuanLySinhVienDB để SELECT. Không yêu cầu gửi mật khẩu thật vào chat.

1. Lưu file. Thay placeholder bằng mật khẩu của máy; giữ mật khẩu thật ngoài bản nộp/Git.

2. Chạy script bằng tài khoản quản trị. Login ở server, user và quyền ở Database.

3. Thử kết nối SSMS bằng sv_app qua TCP, chọn QuanLySinhVienDB, chạy SELECT.

Vì sao cần INSERT ngay hôm nay?

Buổi trước đã có POST thêm sinh viên. Khi chuyển nơi lưu sang SQL Server, chức năng đó phải ghi cùng nguồn dữ liệu với GET. Nếu chỉ cấp SELECT và giữ POST ghi vào mảng, thêm xong sẽ không đọc lại được từ Database.

Điểm dừng kiểm tra:
sv_app SELECT được bảng. Script GRANT có cả SELECT, INSERT; không cấp UPDATE/DELETE vì bài chính chưa cần. Những thao tác sửa dữ liệu minh họa trong SSMS dùng tài khoản quản trị bài học.

Khắc phục nhanh

Login failed: kiểm tra Mixed Mode, đúng login/mật khẩu và Engine đã restart. Cannot open database: kiểm tra tên Database và user mapping. SELECT permission denied: kiểm tra GRANT đúng bảng và Database.

9. Cài Sequelize và bổ sung .env

Terminal ở gốc backend-quan-ly-sinh-vien:

```
pnpm add sequelize@6 tedious
```

dotenv và tsx đã cài từ buổi trước. Không cần pnpm init, không cài @types/sequelize; Sequelize 6 có khai báo kiểu riêng.

PROMPT - sao chép gửi AI

Giữ package.json type:module, pnpm, Express 5, tsx và tsconfig NodeNext hiện có. Tôi đã chạy pnpm add sequelize@6 tedious.
Cập nhật .env.example và hướng dẫn bổ sung .env local: PORT=5000, DB_HOST=127.0.0.1, DB_PORT=1433, DB_NAME=QuanLySinhVienDB, DB_USER=sv_app, DB_PASSWORD dùng placeholder, DB_ENCRYPT=true, DB_TRUST_CERT=true.
Giữ các biến cũ, không tạo PORT trùng. .gitignore bỏ .env/node_modules/dist nhưng giữ .env.example và pnpm-lock.yaml. Không ghi mật khẩu thật vào code. Chưa sửa Service.

```
PORT=5000
DB_HOST=127.0.0.1
DB_PORT=1433
DB_NAME=QuanLySinhVienDB
DB_USER=sv_app
DB_PASSWORD="MAT_KHAU_LOCAL_CUA_BAN"
DB_ENCRYPT=true
DB_TRUST_CERT=true
```

File .env ở cùng cấp package.json, không ở src. DB_PORT là SQL Server, PORT là Express. Đổi .env phải dừng và chạy lại server để nạp chắc chắn. Trust certificate=true chỉ dùng môi trường học local; triển khai thật phải dùng cấu hình chứng chỉ phù hợp.

Điểm dừng kiểm tra:
pnpm-lock.yaml cập nhật; không tạo package-lock.json. File .env.example dùng placeholder, còn .env có mật khẩu local do bạn tự điền.

10. Sinh file kết nối Database

PROMPT - sao chép gửi AI

Chỉ tạo src/config/database.ts, dùng Sequelize 6, TypeScript ESM/NodeNext.
Import "dotenv/config", import { Sequelize } from "sequelize"; export const sequelize là instance dùng chung. Đọc DB_NAME/DB_USER/DB_PASSWORD/DB_HOST từ .env, báo rõ tên biến nếu thiếu nhưng không log mật khẩu. Đọc DB_PORT thành số nguyên 1..65535.
Sequelize options: dialect:"mssql", host/port, logging:console.log. Tedious options đặt trong dialectOptions.options: encrypt từ DB_ENCRYPT, trustServerCertificate từ DB_TRUST_CERT, chuyển string true/false đúng, không dùng Boolean("false").
Không tạo bảng, không sync, không mở HTTP. Trả đầy đủ file có comment tiếng Việt, không thay cấu hình module. Model và script sẽ import ../config/database.js theo ESM.

1. Tạo src/config/database.ts và lưu code.

2. Đọc file: có một instance sequelize được export; config đọc đúng tên biến.

3. Chạy pnpm build. Nếu báo lỗi import/library, gửi cấu hình và lỗi cho AI, giữ NodeNext.

Đối chiếu phần options của Tedious

```
dialectOptions: {
  options: {
    encrypt: process.env.DB_ENCRYPT !== "false",
    trustServerCertificate:
      process.env.DB_TRUST_CERT === "true",
  },
},
```

Đoạn trên chỉ minh họa phần options trong Sequelize constructor; không phải toàn bộ database.ts. Không nối thêm nếu AI đã có cấu hình tương đương.

Điểm dừng kiểm tra:
Build không lỗi. Code không hard-code tài khoản, không dùng cú pháp @sequelize/core của v7 và không gọi sync.

11. Kiểm tra riêng kết nối

PROMPT - sao chép gửi AI

Tạo src/scripts/check-db.ts. Import sequelize từ ../config/database.js. Trong async main(), await sequelize.authenticate(); thành công in "DB connected: QuanLySinhVienDB". try/catch/finally: lỗi báo chẩn đoán không lộ secrets, đặt process.exitCode=1; finally await sequelize.close(). Không mở Express hoặc tạo bảng.
Thêm scripts "check:db":"tsx src/scripts/check-db.ts" vào package.json, giữ nguyên dev="tsx watch src/server.ts", build="tsc", start="node dist/server.js", dependencies và type:module.
Đưa đầy đủ các file sửa, chỉ rõ vị trí, hướng dẫn pnpm build và pnpm check:db.

1. Tạo src/scripts/check-db.ts. Ghép scripts vào package.json, không thay cả file chỉ bằng scripts.

2. Chạy từng lệnh dưới đây trong terminal gốc.

3. Chụp kết quả kết nối thành công rồi mới đi tiếp.

```
pnpm build
pnpm check:db
```

Điểm dừng kiểm tra:
Build thành công, check:db xác thực được đúng Database rồi kết thúc. Kết nối thành công chưa chứng minh Model khớp bảng; bước tiếp theo kiểm tra ánh xạ.

Hai kiểu vòng đời

Script kiểm tra dùng xong thì close để thoát. Server phục vụ API cần giữ kết nối; không gọi close ngay sau authenticate khi HTTP server đang chạy.

12. Tạo Model SinhVien

PROMPT - sao chép gửi AI

Tạo src/models/SinhVien.ts bằng Sequelize 6 + TypeScript ESM. Import sequelize từ ../config/database.js. Class SinhVien extends Model, dùng Model.init và declare thuộc tính để không che getter/setter.
Bảng sẵn có dbo.SinhVien: maSV VARCHAR(5) PRIMARY KEY, hoTen NVARCHAR(150), email NVARCHAR(254), maLop NVARCHAR(50), đều NOT NULL. Map maSV bằng DataTypes.STRING(5), hoTen STRING(150), email STRING(254), maLop STRING(50); MSSQL dùng chuỗi Unicode cho STRING, driver truyền maSV tương thích VARCHAR.
Options tableName:"SinhVien", schema:"dbo", timestamps:false. maSV primaryKey:true, không autoIncrement, không thêm id hoặc createdAt/updatedAt. Không UNIQUE email. Có thể dùng InferAttributes/InferCreationAttributes đúng kiểu; maSV bắt buộc khi tạo.
Giữ src/dtos/sinhVien.dto.ts và interface ISinhVienDTO bốn string như cũ. Giải thích DTO cho dữ liệu trao đổi, Model cho ánh xạ bảng. Không seed trong code, không sync; chỉ tạo Model, chưa thay Service.

1. Tạo src/models/SinhVien.ts. Không ghi đè file DTO bằng class Model.

2. Đối chiếu tên và độ dài trường với SQL; kiểm tra maSV là PK duy nhất.

3. Chạy pnpm build.

Điểm dừng kiểm tra:
Model có bốn trường, đúng tableName và timestamps:false. Không tự sinh id. DTO vẫn export ISinhVienDTO để Controller/Service dùng kiểu phản hồi cũ.

Model.init chỉ khai báo ánh xạ. Database và bảng đã tạo trong SSMS; không dùng Model.init như lệnh tự tạo bảng.

13. Chuyển Service sang Database

Gửi Service, DTO, Model và AppError hiện tại. Nếu có Repository đang chạy, gửi cả file đó và yêu cầu truy vấn đặt ở Repository.

PROMPT - sao chép gửi AI

Thay mảng sinhVienList trong src/services/sinhVien.service.ts bằng Model SinhVien. Giữ tên các hàm; đổi kiểu trả async:
layDanhSachSinhVien(): Promise<ISinhVienDTO[]> -> findAll order maSV ASC.
timSinhVienTheoMa(maSV:string): Promise<ISinhVienDTO|undefined> -> findByPk; null chuyển undefined để Controller cũ xử lý 404.
themSinhVien(input:unknown): Promise<ISinhVienDTO> -> validate rồi create.
Giữ validation cũ: object không null/array; bốn string; trim; maSV /^SV[0-9]{3}$/; hoTen/maLop không rỗng; email /^[^s@]+@[^s@]+.[^s@]+$/. Sai throw AppError(400,...). Thêm giới hạn hoTen<=150, email<=254, maLop<=50; vượt trả 400, không cắt chuỗi.
Chỉ ghi/return bốn trường DTO. Không dùng Express trong Service. Model instance chuyển plain DTO rõ bốn trường. Không dùng any tường minh.
Kiểm tra maSV tồn tại để báo AppError(409,"Mã sinh viên đã tồn tại"). Vẫn catch Sequelize UniqueConstraintError của PK khi create để xử lý hai request cùng mã; không đổi mọi lỗi DB thành 409. Lỗi DB khác chuyển tiếp cho middleware 500.
Không UNIQUE email, không giới hạn mã lớp chỉ 25CT114 trong bài chính. Không giữ POST ghi mảng trong khi GET đọc SQL. Không seed/reset dữ liệu mỗi lần server chạy. Không thêm PUT/DELETE.
Nếu project đã có Repository thì giữ Service validation, đưa findAll/findByPk/create vào Repository, không tạo đường truy vấn thứ hai. Trả đủ các file thay đổi và giải thích vì sao phải await.

Điểm dừng kiểm tra:
Cả ba hàm dùng cùng SQL Server; không còn mảng giả trong đường đi thực tế. Không chạy build riêng ở đây nếu Controller chưa đổi: đổi đồng bộ Controller ngay bước sau rồi build.

14. Controller phải chờ Service

PROMPT - sao chép gửi AI

Đọc Controller/Route/Service hiện tại. Service đã async sau khi chuyển sang SQL Server. Sửa src/controllers/sinhVien.controller.ts để async/await đúng cả ba hàm:
getDanhSachSinhVien -> await layDanhSachSinhVien, 200 {success:true,message:"Lấy danh sách sinh viên thành công",data}.
getSinhVienTheoMa -> đọc req.params.maSV với kiểm tra kiểu phù hợp Express 5; await timSinhVienTheoMa. Có dữ liệu trả 200 {success:true,message:"Tìm sinh viên thành công",data}; undefined trả 404 {success:false,message:"Không tìm thấy sinh viên"}.
taoSinhVien -> await themSinhVien(req.body), 201 {success:true,message:"Thêm sinh viên thành công",data}.
Try/catch và next(error), không gửi response lần hai. Return Promise<void> phù hợp, không any. Không đưa SQL/Model vào Controller.
Giữ Route GET /, GET /:maSV, POST /; server mount /api/v1/sinh-vien một lần. Giữ AppError(400/409), middleware JSON hỏng 400, URL không có 404, lỗi chưa biết 500. Chỉ sửa error middleware nếu thật sự cần, không thay bằng handler làm mọi lỗi thành 500.
Trả đủ file cần sửa, import nội bộ .js; giữ pnpm và NodeNext. Hướng dẫn pnpm build sau khi Service và Controller cập nhật đồng bộ.

1. Áp dụng Service rồi Controller/những file liên quan cùng một chặng.

2. Chạy pnpm build. Nếu còn trả Promise vào JSON, kiểm tra await ở từng hàm Controller.

3. Kiểm tra Route không bị đổi sang /students hoặc /api/products.

Điểm dừng kiểm tra:
Build đạt. DTO/JSON không chứa Sequelize internals hoặc Promise. Các status 200/201/400/404/409 vẫn theo bài trước.

15. Sửa khởi động trong server.ts

PROMPT - sao chép gửi AI

Sửa src/server.ts hiện có để await sequelize.authenticate() trước khi app.listen. Dùng async start(), import ./config/database.js. Khi DB thành công mới log HTTP server chạy; lỗi khởi động log chẩn đoán không lộ secrets, đóng connection và báo exit code khác 0.
Giữ logger, helmet/cors, express.json, GET /, mount /api/v1/sinh-vien, middleware 404 và errorMiddleware đúng thứ tự. Không tạo app.ts chỉ để đổi cấu trúc; không listen hai lần. Không seed/sync lúc khởi động. Không đóng kết nối sau authenticate thành công khi server đang hoạt động. Giữ PORT .env và scripts dev/build/start. Trả server.ts hoàn chỉnh đã ghép, chỉ rõ phần thêm; hướng dẫn pnpm build, pnpm check:db, pnpm dev và pnpm start sau build.

```
pnpm build
pnpm check:db
pnpm dev
```

Để kiểm tra bản build: dừng dev bằng Ctrl+C, sau đó pnpm start. Không chạy dev và start cùng cổng một lúc. Thử GET / và danh sách trên cả cách chạy dev lẫn start để phát hiện import ESM sai.

Điểm dừng kiểm tra:
DB kết nối trước HTTP. / vẫn trả lời chào cũ; /api/v1/sinh-vien trả dữ liệu từ bảng. Log SQL cho thấy SELECT, không có script tự xóa/tạo lại dữ liệu.

Luồng xử lý mới

Postman → Route → Controller (await) → Service → SinhVien Model → Sequelize/Tedious → SQL Server → DTO → Controller trả success/message/data. Nếu có Repository thì tầng đó nằm giữa Service và Model.

16. Dùng lại Collection Postman

1. Mở Postman desktop hoặc web với Desktop Agent. Mở Collection đã làm “Backend Sinh Vien - Chuong 2”; có thể nhân bản thành “Backend Sinh Vien - Buoi 4 SQL Server”.

2. Giữ biến baseUrl=http://localhost:5000 (đổi nếu PORT thực tế khác).

3. Send GET {{baseUrl}}/, GET {{baseUrl}}/api/v1/sinh-vien và GET {{baseUrl}}/api/v1/sinh-vien/SV001.

4. Trong SSMS SELECT cùng bảng, so sánh đủ bốn trường. Thử SV999 chỉ khi mã đó chắc chắn chưa có.

```
{
  "success": true,
  "message": "Tìm sinh viên thành công",
  "data": {
    "maSV": "SV001",
    "hoTen": "Minh Nhật",
    "email": "minhnhat@example.com",
    "maLop": "25CT114"
  }
}
```

Test danh sách giữ hợp đồng cũ

```
pm.test("Danh sach dung hop dong", () => {
  pm.response.to.have.status(200);
  const b = pm.response.json();
  pm.expect(b.success).to.eql(true);
  pm.expect(b.data).to.be.an("array");
  pm.expect(b.data.some(sv => sv.maSV === "SV001"))
    .to.eql(true);
});
```

Điểm dừng kiểm tra:
Test ở Scripts → Post-response của request danh sách, không áp cho cả Collection có nhiều status. HTTP 200 cùng dữ liệu đúng mới đạt; chỉ 200 chưa đủ chứng minh truy vấn đúng bảng.

17. Thêm SV003 và kiểm tra sau restart

Điều kiện: Database chưa có SV003. Kiểm tra bằng GET trước; nếu đã có, dùng mã mới hợp lệ và thay mã đó trong mọi request/test tương ứng, không xóa dữ liệu để chạy lại.

1. Dùng POST {{baseUrl}}/api/v1/sinh-vien. Body → raw → JSON; kiểm tra Content-Type: application/json.

2. Send body dưới đây: kỳ vọng 201, success=true, data.maSV=SV003.

3. GET /api/v1/sinh-vien/SV003: dữ liệu đúng body; trong SSMS SELECT cũng có SV003.

4. Dừng pnpm dev bằng Ctrl+C, chạy lại. Gọi GET SV003: vẫn thấy Ngọc Mai.

5. POST lại cùng SV003: 409; GET danh sách đếm SV003 chỉ một dòng.

```
{
  "maSV": "SV003",
  "hoTen": "Ngọc Mai",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

```
USE QuanLySinhVienDB;
SELECT maSV, hoTen, email, maLop
FROM dbo.SinhVien WHERE maSV = 'SV003';
```

Điểm dừng kiểm tra:
POST lưu vào SQL Server; GET/SSMS đọc lại cùng bản ghi; restart không mất; gửi lại trùng mã trả 409.

Thay đổi cách chạy test so với bài trước: restart không reset Database. T05 “thêm lần đầu” cần mã chưa có. Seed chỉ bổ sung mã thiếu, không xóa các sinh viên đã thêm. Không restart rồi mặc định data trở về hai phần tử.

18. Kiểm tra validation và lỗi

Mỗi ca dùng mã chưa có như SV004; kiểm tra trước bằng GET. Chỉ thay một chi tiết trong body hợp lệ, rồi GET lại sau lỗi để chắc chắn không lưu.

Ca thử	Kỳ vọng
Thiếu hoTen / tên chỉ khoảng trắng	400; success=false; mã thử không xuất hiện.
maSV là số 4 / ABC004	400; không tự chuyển sang chuỗi/mã hợp lệ.
Email khong-phai-email	400; không lưu.
Thiếu hoặc rỗng maLop	400; không lưu.
hoTen dài hơn 150 ký tự	400; không lưu, không cắt chuỗi.
JSON hỏng {"maSV":"SV004"	400 JSON; không trả HTML lỗi dài.
Mã đã tồn tại	409; vẫn chỉ một bản ghi.
GET mã không có / URL không có	404; đọc message phân biệt hai nguyên nhân.

PROMPT - sao chép gửi AI

Tôi đã chuyển bài quản lý sinh viên sang SQL Server. Đây là một request POST bị từ chối: [method, URL, body, response]. Kỳ vọng: 400 hoặc 409 đúng quy tắc, success=false, message rõ; không lưu bản ghi lỗi/trùng.
Hãy đối chiếu thực tế với yêu cầu trước. Viết Postman test cho status/success/message đặt ở request này; thêm request GET kiểm tra mã lỗi không tồn tại hoặc mã trùng chỉ có một. Không khẳng định chỉ response POST đã chứng minh không ghi dữ liệu. Không bỏ validation/PK để test pass.

Điểm dừng kiểm tra:
Các ca cũ vẫn đúng sau chuyển Database. Lỗi DB chưa biết phải trả 500 với “Lỗi hệ thống”, không lộ SQL/mật khẩu trong response và không bị nhầm thành mã trùng.

19. Test độ bền dữ liệu có ý nghĩa

Đặt test dưới đây trong request GET SV003. Nó phải pass cả trước và sau khi restart; nếu dùng mã mới thay SV003 thì chỉnh kỳ vọng tương ứng.

```
pm.test("SV003 van duoc luu", () => {
  pm.response.to.have.status(200);
  const b = pm.response.json();
  pm.expect(b.success).to.eql(true);
  pm.expect(b.data.maSV).to.eql("SV003");
  pm.expect(b.data.hoTen).to.eql("Ngọc Mai");
  pm.expect(b.data.email).to.eql("ngocmai@example.com");
  pm.expect(b.data.maLop).to.eql("25CT114");
});
```

Thử thay đổi trực tiếp nguồn dữ liệu

SSMS bằng tài khoản có quyền UPDATE, sửa SV002 như dưới. Send lại GET SV002 mà không sửa Service. Xác nhận tên mới; bản seed không được ghi đè tên này khi khởi động lại.

```
USE QuanLySinhVienDB;
UPDATE dbo.SinhVien
SET hoTen = N'Lan Anh (đã cập nhật)'
WHERE maSV = 'SV002';
SELECT * FROM dbo.SinhVien WHERE maSV = 'SV002';
```

Sau minh họa có thể dùng tài khoản quản trị khôi phục tên Lan Anh nếu cần chạy test tên cũ. Ghi rõ điều kiện dữ liệu trong báo cáo.

Quan sát FAIL có chủ đích

Tạm đổi kỳ vọng status GET từ 200 sang 201, Send để thấy FAIL, rồi khôi phục 200. Đây là kiểm tra cơ chế test, không phải lỗi chức năng. Không thay kỳ vọng đúng theo response sai để có PASS.

Điểm dừng kiểm tra:
Có bằng chứng POST → SELECT → restart → GET. Có bằng chứng SSMS sửa bản ghi làm API thay đổi. Bạn giải thích được khác biệt RAM và lưu bền vững.

20. Mở rộng: Lớp 1-N Sinh viên

Ví dụ quan hệ nối với bài cũ là **Lop(maLop, tenLop)** và **SinhVien(maSV, hoTen, email, maLop)**. Một lớp có nhiều sinh viên. maLop ở SinhVien tham chiếu Lop.maLop, không đổi tên trường DTO.

Quy tắc mới ở phần mở rộng: trước đây maLop chỉ cần chuỗi không rỗng. Khi thêm FK, maLop phải có trong bảng Lop. Phải công bố quy tắc này và xử lý POST lớp không tồn tại bằng 400, không để nó thành 500 khó hiểu.

PROMPT - sao chép gửi AI

Mở rộng project quản lý sinh viên đã chạy SQL Server:
Tạo sql/04-lop-va-quan-he.sql: dbo.Lop có maLop NVARCHAR(50) PK, tenLop NVARCHAR(150) NOT NULL. Trước khi thêm FK, tạo bản ghi Lop cho tất cả maLop DISTINCT đang có trong SinhVien; có thể dùng tenLop=maLop làm tên tạm để tôi cập nhật. Không xóa sinh viên/lớp hoặc thay maLop cũ. Sau khi đối chiếu dữ liệu, thêm FK SinhVien.maLop -> Lop.maLop. Kiểu cột phải khớp. Không cascade delete.
Tạo src/models/Lop.ts và src/models/index.ts. Khai báo Lop.hasMany(SinhVien,{foreignKey:"maLop",as:"sinhViens"}); SinhVien.belongsTo(Lop,{foreignKey:"maLop",as:"lop"}). ESM import .js, timestamps:false, tableName/schema đúng. Import qua index để quan hệ chạy, tránh circular import.
Giữ response API chính với bốn trường DTO. Dùng script riêng để findAll include alias lop và in dữ liệu, chưa thêm lop vào API cũ. Cập nhật Service themSinhVien: sau validation kiểm tra Lop tồn tại, thiếu trả AppError(400,"Mã lớp không tồn tại"); xử lý ForeignKeyConstraintError liên quan maLop thành 400 nếu có tranh chấp, không coi mọi lỗi là lỗi lớp. Cấp SELECT bảng Lop cho sv_app.
Đưa từng file, thứ tự SQL -> quyền -> code -> pnpm build -> test. Không sync/drop và không tự thêm CRUD lớp.

Điểm dừng kiểm tra:
Dữ liệu cũ vẫn đọc được; include cho đúng lớp. POST mã lớp không có trả 400; POST mã lớp có vẫn 201. Alias/association trong ORM và FK trong SQL là hai phần khác nhau.

21. Khi lỗi: thu thập bằng chứng

PROMPT - sao chép gửi AI

Tôi đang ở bước [số] bài quản lý sinh viên Buổi 4.
Môi trường: pnpm, Express 5, TypeScript ESM/NodeNext, tsx, Sequelize 6, SQL Server. Lệnh/request: [dán]. Mong đợi: [ghi]. Thực tế/lỗi nguyên văn: [dán]. File hiện tại liên quan: [dán và ghi đường dẫn]. Cấu hình DB đã che mật khẩu: [host/port/name]. SSMS TCP với sv_app đã SELECT được chưa: [có/chưa]. Hãy nêu nguyên nhân và cách xác minh trước, sửa nhỏ nhất đúng file. Giữ DTO bốn trường, URL /api/v1/sinh-vien, success/message/data và status cũ. Không chuyển npm/CommonJS, bỏ strict, xóa DB hoặc sync force/alter. Trả file đầy đủ nếu sửa và lệnh pnpm kiểm tra lại. Chưa khẳng định đã chạy SQL Server nếu không có kết quả thực tế.

Biểu hiện	Kiểm tra trước
Build ESM/import lỗi	Đuôi .js ở import nội bộ, tên export, type:module và NodeNext.
Login failed / timeout	Mixed Mode, login, Engine, đúng TCP/port, SSMS cùng đường kết nối.
Invalid object/column	DB_NAME, dbo.SinhVien, tableName, timestamps:false.
POST 201 nhưng GET không thấy	POST còn ghi mảng? Có await create? GET đang dùng DB nào?
Restart mất SV003	Còn mảng hay seed/reset ở startup? DB_NAME có đổi không?
Trùng mã thành 500	AppError/UniqueConstraintError được map đúng chưa?
Request trả JSON sai	Controller đã await cả ba Service chưa?
Test thêm lần đầu báo 409	Mã đã lưu từ lần trước; restart không reset DB.

Điểm dừng kiểm tra:
Sau sửa, chạy lại ca bị lỗi và các chức năng cũ. Ghi lại nguyên nhân thật; nếu chưa gặp lỗi thì ghi chưa phát sinh lỗi, không tạo bằng chứng giả.

22. Nộp bài và tự giải thích

Hồ sơ nộp

• Source src, sql, package.json, pnpm-lock.yaml, tsconfig.json, .env.example, .gitignore; không node_modules/dist/.env/mật khẩu thật. Script login trong bài nộp dùng placeholder.

• Collection Postman cũ đã bổ sung ca lưu bền vững và test; baseUrl đúng máy chạy.

• README: cấu hình SQL Server, thứ tự script, quyền sv_app, lệnh pnpm build/check:db/dev/start và điều kiện trước từng test.

• Bảng kết quả cùng ảnh: check:db, SELECT bảng, POST thành công, GET sau restart, 409 trùng và 400 validation.

• Nhật ký prompt ngắn: yêu cầu, kiểm tra, điều hiểu thêm; ghi phần bạn đã điều chỉnh.

Tiêu chí	Bằng chứng
Đúng project cũ	pnpm/ESM, file sinhVien.*, DTO và URL cũ.
Đọc SQL	GET danh sách/theo mã đúng SELECT.
Ghi SQL	POST 201; SELECT thấy bản ghi.
Lưu bền vững	Restart không mất sinh viên đã thêm.
Nghiệp vụ giữ đúng	400 sai dữ liệu, 404 không có, 409 trùng; không lưu lỗi.
Quan hệ mở rộng	Lop 1-N SinhVien; quy tắc lớp tồn tại được công bố.

Tự trả lời bằng lời của bạn

DTO và Model khác nhau thế nào? Vì sao Controller cần await sau khi đổi Service? Vì sao restart không dùng để reset test nữa? PK bảo vệ trùng mã thế nào? Tại sao POST và GET phải đọc/ghi cùng nguồn?

Tài liệu đối chiếu

Bài thực hành Chương 2 Vibe Coding và Postman V2: project quản lý sinh viên, pnpm/ESM, bốn trường DTO, các API GET/POST và test. Chương trình Liên thông Buổi 4: kết nối SQL Server, Sequelize và quan hệ Model; ví dụ quan hệ được chuyển thành Lớp–Sinh viên để nối đúng project.

Tra cứu chính thức: sequelize.org/docs/v6/getting-started/; Sequelize 6 TypeScript. Code do AI sinh cần build và kiểm tra SQL Server trên máy của bạn.